CUDA-sDBSCAN: GPU Accelerated Density Based Clustering With Random Projections

Hugo Phibbs and Ninh Pham

***Abstract*—**

I. Introduction

DBSCAN [2] is clustering based algorithm with many applications in areas such as...

II. Related Work

III. Preliminaries

A. DBSCAN

Perhaps comment that the below is a naive implementation of DBSCAN?

In a nutshell, DBSCAN [1] is an algorithm to cluster related data points together based on a given distance metric. The algorithm is composed of two key processes: the identification of core points and the formation of clusters. For the sake of explanation consider a set of data points $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$, where $dim(\mathbf{x}_i) = d$. The identification of core points can be completed as follows:

1) For each data point $q \in \mathbf{X}$ find the distance $dist(\mathbf{x}, \mathbf{q})$ to all other data points $\mathbf{x} \in \mathbf{X}$. If $dist(\mathbf{x}, \mathbf{q})$ is less than the maximum clustering distance ϵ, add $\mathbf{x}$ to the neighbourhood set of $\mathbf{q}$, denoted by $B_\epsilon(\mathbf{q})$
2) For each $\mathbf{q} \in \mathbf{X}$, if $|B_\epsilon(\mathbf{q})|$ is greater than or equal to a threshold $minPts$, the point $\mathbf{q}$ is a *core point*. Add the tuple $(\mathbf{q}, B_\epsilon(\mathbf{q}))$ to the core point set C.

The pseudo code for the identification of key points is shown in algorithm 1.

Algorithm 1 DBSCAN, classification of core points

Input: Data points $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$, ϵ, $minPts$
Output: Set containing core points and their ϵ-neighbourhoods
Initialize $C = \emptyset$
for each data point $\mathbf{q} \in \mathbf{X}$ **do**
    Initialize $B_\epsilon(\mathbf{q}) = \emptyset$
    for each data point $\mathbf{x} \in \mathbf{X}$ **do**
        Compute $dist(\mathbf{x}, \mathbf{q})$
        if $dist(\mathbf{x}, \mathbf{q}) < \epsilon$ **then**
            Add $\mathbf{x}$ to $B_\epsilon(\mathbf{q})$
        end if
    end for
    if $|B_\epsilon(\mathbf{q})| \geq minPts$ **then**
        Add $(\mathbf{q}, B_\epsilon(\mathbf{q}))$ to C
    end if
end for

Once core points are identified, the data set can be partitioned into clusters. A cluster is formed by considering all points that are within *direct* or *transitive* ϵ distance from each other. Given the identification of core points and their neighbourhoods, clusters can be formed using algorithm 2. The result of the DBSCAN algorithm is a graph G, where each connected component is a cluster of the data set $\mathbf{X}$

Algorithm 2 DBSCAN

Inputs: $\mathbf{X}, \epsilon, minPts$ the set $C = \{(\mathbf{q}, B_\epsilon(\mathbf{q})) \mid \mathbf{q} \text{ is core}\}$
$G \leftarrow$ initialize empty graph
for each $\mathbf{q} \in C$ **do**
    Add an edge (and possibly a vertex or vertices) in G from $\mathbf{q}$ to all *core* points in $B_\epsilon(\mathbf{q})$
    Add an edge (and possibly a vertex) in G from $\mathbf{q}$ to *non-core* points $\mathbf{x} \in B_\epsilon(\mathbf{q})$ if $\mathbf{x}$ is not connected
end for
return connected components of G

B. sDBSCAN

Comment here about the scalability of DBSCAN, and how this is addressed with sDBSCAN.

sDBSCAN[6] is an implementation of DBSCAN based on using random projections to rapidly speed up the ϵ-neighbourhood search for each particular data point.

Besides the given DBSCAN inputs of $\mathbf{X}$, ϵ, and $minPts$, sDBSCAN adds a few more parameters - these are described in table I.

Parameter	Description
$\mathbf{S}$	A set of D random vectors with the same dimension as points in $\mathbf{X}$. Each random vector has coordinates sampled from the standard normal distribution. Formally: $\mathbf{S} = \{\mathbf{r}^i \mid \mathbf{r}^i \in \mathbb{R}^d,\ \mathbf{r}^i \sim \mathcal{N}(0,1),\ i \in [D]\}$
k	The number of closest/furthest random vectors from $\mathbf{S}$ to store for each query point $\mathbf{q} \in \mathbf{X}$
m	The number of closest/furthest query points from $\mathbf{X}$ to store for each random vector $\mathbf{r} \in \mathbf{S}$

TABLE I
Definitions and descriptions of sDBSCAN's parameters

The sDBSCAN algorithm can be summarised into a few key steps: pre-processing, identification of key points, and formation of clusters. The pre-processing steps are as follows:s

1) For each random vector $\mathbf{r}_i \in \mathbf{S}$, calculate it's projection to each query point $\mathbf{q} \in \mathbf{X}$.

2) For each query point $\mathbf{q} \in \mathbf{X}$, using the projections store its top k closest and furthest random vectors.
3) For each random vector $\mathbf{r}_i \in \mathbf{S}$, using the projections, store its top m closest and furthest data set points.
4) Use the random projections to form an approximate ϵ-neighbourhood set $\tilde{B}_\epsilon(\mathbf{q})$ for each data-set point $\mathbf{q}$.
5) Perform DBSCAN on the set of all approximate $\tilde{B}_\epsilon(\mathbf{q})$ sets for $\mathbf{q}$ identified as core.

The complete pseudocode is shown in algorithm 3.

Algorithm 3 Preprocessing

Inputs: $\mathbf{X} \subset \mathcal{S}^{d-1}$, D random vectors $\mathbf{r}_i$, k, $m = O(minPts)$
for each $\mathbf{q} \in \mathbf{X}$, compute and store top-$k$ closest and top-k furthest vectors $\mathbf{r}_i$ to $\mathbf{q}$.
for each random vector $\mathbf{r}_i$, compute and store top-m closest and top-m furthest points to $\mathbf{r}_i$.

Fig. 1.

Algorithm 4 Finding core points and their neighborhoods

Inputs: $\mathbf{X} \subset \mathcal{S}^{d-1}$, D random vectors $\mathbf{r}_i$, k, ε, $m = O(minPts)$
Initialize an empty set $\widetilde{B}_\varepsilon(\mathbf{q})$ for each $\mathbf{q} \in \mathbf{X}$
for each $\mathbf{q} \in \mathbf{X}$ **do**
  for each $\mathbf{r}_i$ from top-k closest random vectors of $\mathbf{q}$ **do**
    for each $\mathbf{x}$ from top-m closest points of $\mathbf{r}_i$ **do**
      if $dist(\mathbf{x}, \mathbf{q}) \leq \varepsilon$ **then**
        Insert $\mathbf{x}$ into $\widetilde{B}_\varepsilon(\mathbf{q})$ and insert $\mathbf{q}$ into $\widetilde{B}_\varepsilon(\mathbf{x})$
  for each $\mathbf{r}_i$ from top-k furthest random vectors of $\mathbf{q}$ **do**
    for each $\mathbf{x}$ from top-m furthest points of $\mathbf{r}_i$ **do**
      if $dist(\mathbf{x}, \mathbf{q}) \leq \varepsilon$ **then**
        Insert $\mathbf{x}$ into $\widetilde{B}_\varepsilon(\mathbf{q})$ and insert $\mathbf{q}$ into $\widetilde{B}_\varepsilon(\mathbf{x})$
for each $\mathbf{q} \in \mathbf{X}$ **do**
  if $|\widetilde{B}_\varepsilon(\mathbf{q})| \geq minPts$ **then**
    Identify $\mathbf{q}$ as the core point
    Store $\widetilde{B}_\varepsilon(\mathbf{q})$ as an approximate result of $B_\varepsilon(\mathbf{q})$
    Store $dist(\mathbf{x}, \mathbf{q})$ for each $\mathbf{x} \in \widetilde{B}_\varepsilon(\mathbf{q})$ {*Only for sOPTICS*}

Fig. 2.

Algorithm 5 sDBSCAN

Inputs: $\mathbf{X} \subset \mathcal{S}^{d-1}$, D random vectors $\mathbf{r}_i$, ε, $minPts$
Call Algorithm 3 for preprocessing with $m = O(minPts)$
Call Algorithm 4 to find the set
$C = \{(\mathbf{q}, \widetilde{B}_\varepsilon(\mathbf{q})) \mid \mathbf{q}$ is identified as core$\}$
Call DBSCAN (Alg. 1) given the output C from Algorithm 4

Fig. 3.

C. GPU Programming with CUDA

A GPU (Graphical Processing Unit) is based around the idea of having using hundreds to thousands of independent cores to complete tasks in a parallelised manner. This strategy excels in areas such as linear algebra, sorting, and differential equations [5]. To ease the development of applications that utilize a GPU, NVIDIA, a major manufacturer of GPU chips, has created CUDA [3] to provide an API for GPU programming.

This sentence could be reworded

.

For the purposes of this paper, we will look at the typical architecture of a NVIDIA GPU, then see how this relates to the programming model of CUDA.

The CUDA programming model

can it be called a model?

abstracts the low levels of programming directly onto a GPU with a few key concepts. At the highest level there is a *kernel grid*, this executes a single set of instructions called a *kernel* across the GPU. A grid is sub components called *thread blocks* these execute a number of individual kernel instructions - called *threads* - across one SM. A single *thread* is executed on one CUDA core - this is the lowest level of programming abstraction with CUDA. Figure 4 explains these concepts.

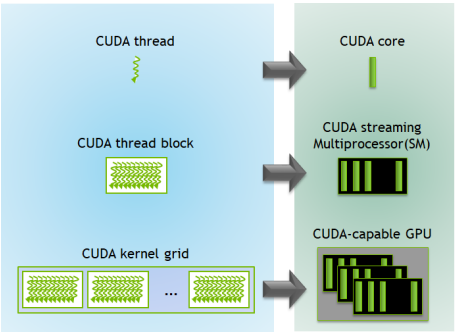


Fig. 4. Simplified overview of the CUDA programming model [4]

Threads are executed in groups called *warps*. To maximise occupancy of an SM while a block is being executed, the size of a block should be a multiple of 32

find a reference for this

.

IV. CUDA-sDBSCAN

V. Experiment

VI. Conclusion

References

[1] Ricardo Campello, Peer Kröger, Joerg Sander, and Arthur Zimek. Density-based clustering. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 10, 10 2019.
[2] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, KDD'96, page 226–231. AAAI Press, 1996.
[3] NVIDIA. Cuda toolkit, 2023.
[4] NVIDIA Developer. Cuda refresher: The cuda programming model, 2023. Accessed: 2024-06-19.
[5] John D. Owens, Mike Houston, David Luebke, Simon Green, John E. Stone, and James C. Phillips. Gpu computing. *Proceedings of the IEEE*, 96(5):879–899, 2008.
[6] Haochuan Xu and Ninh Pham. Scalable density-based clustering with random projections, 2024.